



UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA

Métricas de Evaluación

EIN087B - Ciencia de Datos
Paralelo 701

Profesor: Jorge Portilla G.
jorge.portilla@usm.cl

Departamento de Electrónica e Informática
Universidad Técnica Federico Santa María
Concepción, Chile

Validación cruzada con conjunto de prueba (Hold-out)

Algoritmo: Validación cruzada con conjunto de prueba

1. Dividir aleatoriamente S en S_{train} (por ejemplo, el 75 % de los datos) y S_{cv} (el 25 % restante). Donde S_{cv} es el conjunto de validación cruzada.
2. Entrenar cada modelo M_i en S_{train} solo para obtener la hipótesis f_i .
3. Seleccionar y mostrar la hipótesis f_i con el menor error $\hat{S}_{\text{cv}}(f_i)$ en el conjunto de validación cruzada.

```
1 import numpy as np
2 from sklearn.linear_model import LinearRegression
3 from sklearn.model_selection import train_test_split
4 from sklearn.metrics import mean_squared_error
5
6 np.random.seed(42)
7 X = np.random.rand(100, 1)
8 epsilon = np.random.randn(100, 1)
9 Y = 2 + 3 * X + epsilon
10
11 # CV Hold-Out (80% train, 20% test)
12 X_train, X_test, Y_train, Y_test =
13     train_test_split(X, Y, test_size=0.2, random_state=42)
```

Algoritmo: Validación Cruzada K-Fold

1. Dividir aleatoriamente S en K subconjuntos de M/K ejemplos de entrenamiento cada uno. Llamemos a estos subconjuntos $S_1, \dots, S_K$.
2. Para cada modelo M_i hacer:
 - 1) Para $j = 1, \dots, K$ hacer:
 - 1) $f_{ij} \leftarrow A_i(S \setminus S_j)$ (entrenar con todos los datos excepto S_j)
 - 2) $\hat{S}_j(f_{ij}) = \frac{1}{|S_j|} \sum_{(x_m, y_m) \in S_j} \ell(y_m, f_{ij}(x_m))$ (calcular el error sobre el conjunto de validación)
 - 2) $\hat{M}_i = \frac{1}{K} \sum_{k=1}^K \hat{S}_k(f_{ik})$ (calcular el promedio del error de validación sobre los k pliegues)
3. Seleccionar el modelo M_i con el menor $\hat{M}_i$.
4. $f \leftarrow A_i(S)$ (entrenar una hipótesis con todos los datos según el modelo M_i).
5. Salida: f

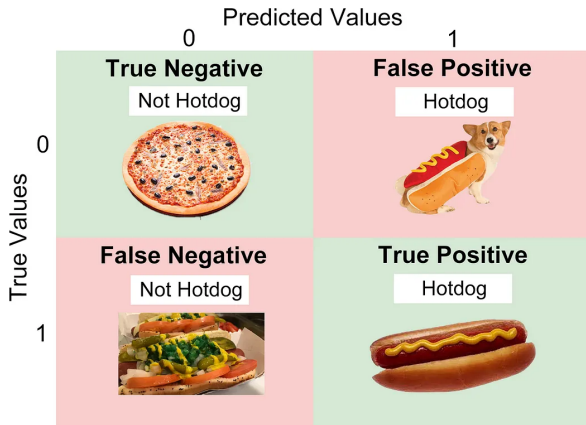
► A_i es el algoritmo de aprendizaje según el modelo M_i .

K-fold cross validation

```
1 import numpy as np
2 from sklearn.linear_model import LinearRegression
3 from sklearn.model_selection import KFold
4 from sklearn.metrics import mean_squared_error
5
6 X = np.random.rand(100, 1)
7 epsilon = np.random.randn(100, 1)
8 Y = 2 + 3 * X + epsilon
9
10 kf = KFold(n_splits=5, shuffle=True, random_state=42)
11 mse_scores = []
12
13 for train_idx, test_idx in kf.split(X):
14     X_train, X_test = X[train_idx], X[test_idx]
15     Y_train, Y_test = Y[train_idx], Y[test_idx]
16
17     reg = LinearRegression()
18     reg.fit(X_train, Y_train)
19     Y_pred = reg.predict(X_test)
20
21     mse = mean_squared_error(Y_test, Y_pred)
22     mse_scores.append(mse)
23
24 print(f"Error MSE promedio K-Fold: {np.mean(mse_scores):.2f}")
```

Técnicas de evaluación de algoritmos de clasificación

- ▶ ...en general, se utilizan métricas que puedan mostrar:
 - **Un número de éxitos o fallas del algoritmo** (proviene de clasificación binaria). En este caso, se usan las siguientes definiciones:
 - **Verdadero Positivo (TP)**: El algoritmo detecta una situación real (existe en la ground-truth y en los resultados del algoritmo).
 - **Falso Negativo (FN)**: No se detectó una situación real (existe sólo en la ground-truth).
 - **Falso Positivo (FP)**: El algoritmo detecta una situación irreal (existe sólo en los resultados del algoritmo).
 - **Verdadero Negativo (TN)**: Elemento que no aparece ni en la groundtruth ni en los resultados del algoritmo.



Número de éxitos o fallas del algoritmo

```
1 from sklearn.metrics import confusion_matrix
2
3 y_true = [1, 0, 1, 1, 0, 0, 1, 0]
4
5 y_pred = [1, 0, 1, 0, 0, 1, 1, 0]
6
7 # Matriz de confusi n: [[TN, FP], [FN, TP]]
8 cm = confusion_matrix(y_true, y_pred)
9
10 TN, FP, FN, TP = cm.ravel()
11
12 print(f"TP: {TP}, TN: {TN}, FP: {FP}, FN: {FN}")
```


- ▶ TP, FP, TN, FN también pueden dar más información para caracterizar a los algoritmos:

- **Precision** = $\frac{TP}{TP + FP}$, tasa de aciertos, contra aciertos y errores del algoritmo¹.

- **Sensitivity** = $\frac{TP}{TP + FN}$, tasa de aciertos, contra todo lo que se debía detectar².

- **Specificity** = $\frac{TN}{FP + TN}$, tasa de elementos correctamente no detectados, contra todo lo que no debía generar respuesta.

- **F-score** = $2 * \frac{\text{Precision} * \text{Sensitivity}}{\text{Precision} + \text{Sensitivity}}$, media armónica entre Precision y Sensitivity.

- **Acc:** = $\frac{TN + TP}{FP + FN + TN + TP}$

¹El objetivo es medir la capacidad del modelo para no identificar erróneamente una señal negativa como positiva.

²Este criterio mide la capacidad del modelo para encontrar todas las señales positivas en el conjunto de datos.

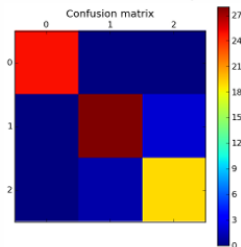
```
1 from sklearn.metrics import confusion_matrix
2
3 y_true = [1, 0, 1, 1, 0, 0, 1, 0]
4 y_pred = [1, 0, 1, 0, 0, 1, 1, 0]
5
6 confusion_matrix(y_true, y_pred).ravel()
7
8
9 precision = TP / (TP + FP)
10 sensitivity = TP / (TP + FN)    #recall
11 specificity = TN / (TN + FP)
12 f_score = 2 * (precision * sensitivity) / (precision + sensitivity)
13 accuracy = (TP + TN) / (TP + TN + FP + FN)
14
15 print(f"Precision: {precision}")
16 print(f"Sensitivity: {sensitivity}")
17 print(f"Specificity: {specificity}")
18 print(f"F-score: {f_score}")
19 print(f"Accuracy: {accuracy}")
```

```
1 from sklearn.metrics import (precision_score, recall_score,
2                               f1_score, accuracy_score,
3                               confusion_matrix)
4
5 y_true = [1, 0, 1, 1, 0, 0, 1, 0]
6 y_pred = [1, 0, 1, 0, 0, 1, 1, 0]
7
8 precision = precision_score(y_true, y_pred)
9 sensitivity = recall_score(y_true, y_pred)
10 f_score = f1_score(y_true, y_pred)
11 accuracy = accuracy_score(y_true, y_pred)
12
13 TN, FP, FN, TP = confusion_matrix(y_true, y_pred).ravel()
14 specificity = TN / (TN + FP)
15
16 print(f"Precision: {precision:.2f}")
17 print(f"Sensitivity: {sensitivity:.2f}")
18 print(f"Specificity: {specificity:.2f}")
19 print(f"F-score: {f_score:.2f}")
20 print(f"Accuracy: {accuracy:.2f}")
```

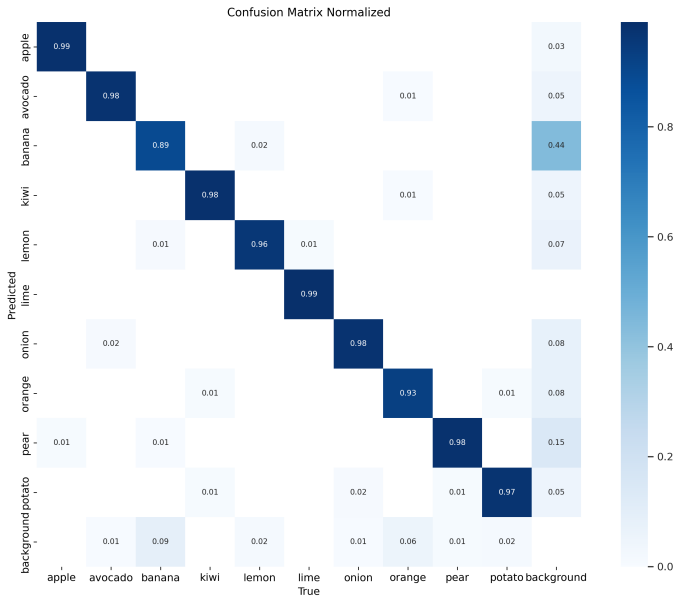
Evaluación: Matriz de Confusión (Confusion Matrix)

- Es una herramienta de visualización típicamente usada en aprendizaje supervisado (muestras de entrenamiento etiquetadas).
- Cada columna de la matriz representa las instancias de clases predichas (e.g., la salida de un algoritmo).
- Cada fila representa las verdaderas instancias de la clase (e.g., etiquetas de las muestras, conocidas a priori).

		prediction outcome		
		p	n	total
actual value	p'	True Positive	False Negative	p'
	n'	False Positive	True Negative	N'
total		P	N	



Ejemplo de Matriz de confusión



Matriz de confusión

```
1 from sklearn.metrics import (precision_score, recall_score,
2                               f1_score, accuracy_score,
3                               confusion_matrix)
4
5 y_true = [1, 0, 1, 1, 0, 0, 1, 0]
6 y_pred = [1, 0, 1, 0, 0, 1, 1, 0]
7
8 precision = precision_score(y_true, y_pred)
9 sensitivity = recall_score(y_true, y_pred)
10 f_score = f1_score(y_true, y_pred)
11 accuracy = accuracy_score(y_true, y_pred)
12
13 cm = confusion_matrix(y_true, y_pred)
14 TN, FP, FN, TP = cm.ravel()
15
16 plt.figure(figsize=(6, 4))
17 sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=False,
18             xticklabels=['Predicted 0', 'Predicted 1'],
19             yticklabels=['Actual 0', 'Actual 1'])
20 plt.xlabel("Predicted")
21 plt.ylabel("Actual")
22 plt.title("Confusion Matrix")
23 plt.show()
```

- ▶ Es fácil ver si el sistema se confunde entre dos o más clases.
 - ▶ Cuando un conjunto de datos es desbalanceado (número de muestras en diferentes clases varía mucho) la tasa de error del clasificador no es representativa del verdadero rendimiento del clasificador.
-

- ▶ e.g. clase A: 990 muestras - clase B: 10 muestras: **¿Qué pasa con la tasa de error si acierto en todas con A y en ninguna con B?.acierto**

- ▶ **Receiver Operating Characteristic (ROC)**, o curva ROC, es una gráfica que ilustra el rendimiento de un clasificador binario, ante la variación de su umbral.
- ▶ Se crea graficando la **TPR** (true positive rate) vs la **FPR** (false positive rate), en distintos umbrales.

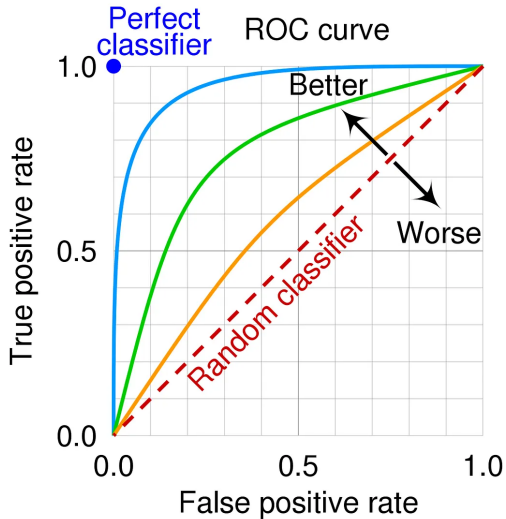
Definiciones Clave

- ▶ **TPR (True Positive Rate):**

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}} = \text{Sensibilidad}$$

- ▶ **FPR (False Positive Rate):**

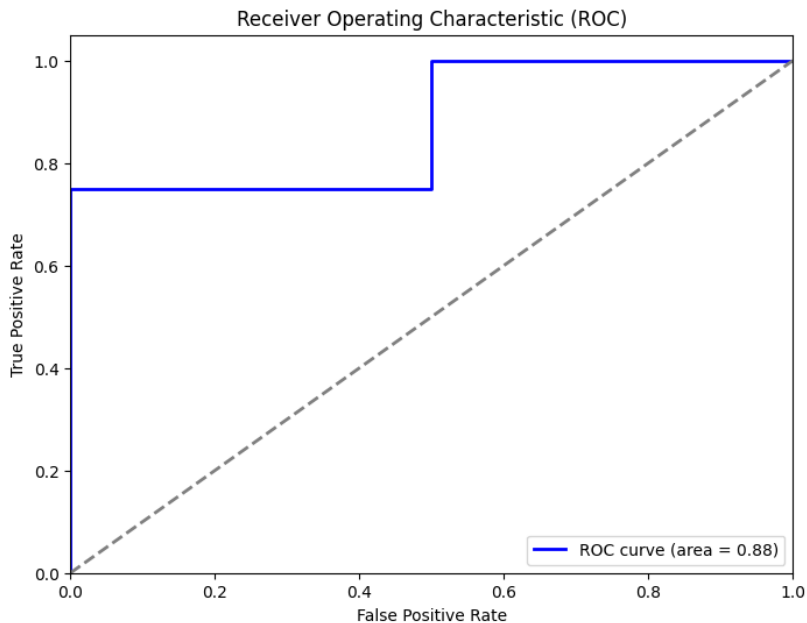
$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}} = 1 - \text{Especificidad}$$



Ejemplo Curva ROC

```
1 import matplotlib.pyplot as plt
2 from sklearn.metrics import roc_curve, auc
3
4 y_true = [1, 0, 1, 1, 0, 0, 1, 0]
5 y_scores = [0.9, 0.2, 0.85, 0.3, 0.1, 0.7, 0.8, 0.4]
6
7
8 fpr, tpr, thresholds = roc_curve(y_true, y_scores)
9
10
11 roc_auc = auc(fpr, tpr)
12
13 plt.figure(figsize=(8, 6))
14 plt.plot(fpr, tpr, color='blue',
15         lw=2, label=f'ROC curve (area = {roc_auc:.2f})')
16 plt.plot([0, 1], [0, 1], color='gray', lw=2, linestyle='--')
17 plt.xlim([0.0, 1.0])
18 plt.ylim([0.0, 1.05])
19 plt.xlabel('False Positive Rate')
20 plt.ylabel('True Positive Rate')
21 plt.title('Receiver Operating Characteristic (ROC)')
22 plt.legend(loc="lower right")
23 plt.show()
```

Ejemplo Curva ROC



Ejemplo de GridSearch

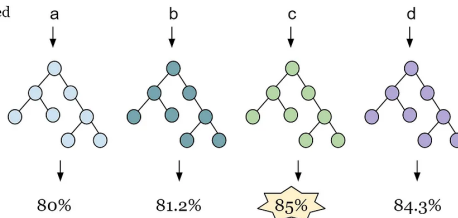
- Parameters are found by training the model.
- Hyperparameters are set by the data scientist before training

```
Test_Hyperparameters = [a, b, c, d]
```

Hyperparameter used
to make the model

model

Accuracy on test set



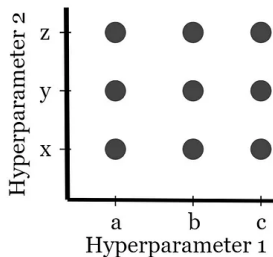
³<https://medium.com/@jackstalfort/hyperparameter-tuning-using-grid-search-and-random-search-f8750a464b35>

Grid Search

Pseudocode

```
Hyperparameter_One = [a, b, c]
```

```
Hyperparameter_Two = [x, y, z]
```

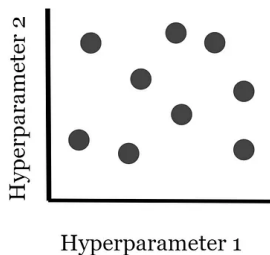


Random Search

Pseudocode

```
Hyperparameter_One = random.num(range)
```

```
Hyperparameter_Two = random.num(range)
```



```
https://scikit-learn.org/stable/modules/generated/sklearn.  
linear\_model.LogisticRegression.html
```



UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA

Métricas de Evaluación

EIN087B - Ciencia de Datos
Paralelo 701

Profesor: Jorge Portilla G.
`jorge.portilla@usm.cl`

Departamento de Electrónica e Informática
Universidad Técnica Federico Santa María
Concepción, Chile